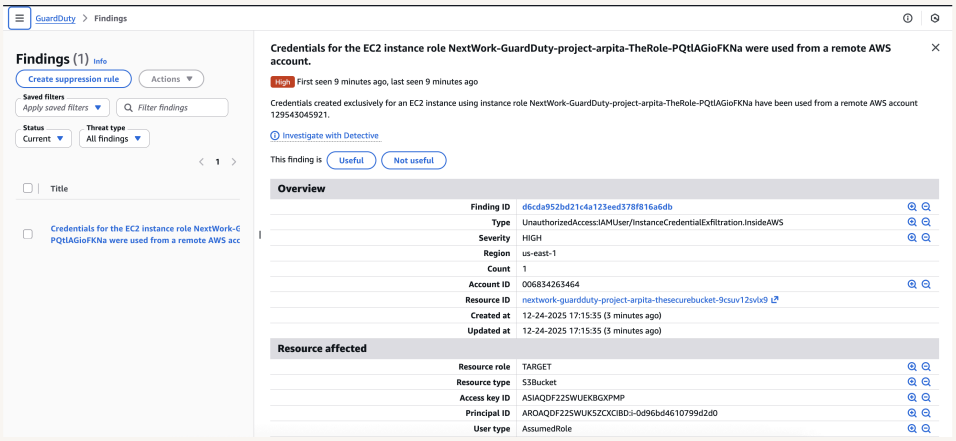
nextwork.org

Threat Detection with GuardDuty

CH Arpita Chowdhury



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Introducing Today's Project!

Tools and concepts

The services I used were AWS CloudFormation, Amazon EC2, Amazon S3, AWS IAM, AWS CloudShell, AWS CLI, Amazon GuardDuty, and GuardDuty Malware Protection for S3. Key concepts I learnt include deploying infrastructure as code, understanding common web vulnerabilities (SQL injection and command injection), credential exfiltration and misuse, how attackers pivot using stolen IAM role credentials, threat detection and anomaly-based findings in GuardDuty, incident analysis using security findings, and proactive malware detection using test files to validate cloud security controls.

Project reflection

This project took me approximately 3 hours to complete. The most challenging part was understanding how the different attack steps connected together from exploiting the web application to observing how GuardDuty detected the resulting credential misuse and malware activity. It was most rewarding to see GuardDuty successfully generate high-severity findings in real time, confirming that the security controls were working and that I could clearly trace attacker behavior from initial compromise to detection.

placeholder

I did this project today to gain hands-on, real-world experience with cloud threat detection and to move beyond theory into actually seeing how attacks are detected in AWS.

CH

Arpita Chowdhury
NextWork Student

nextwork.org

My goal was to understand how credential compromise, unauthorized access, and malware activity look from a defender’s point of view and how Amazon GuardDuty surfaces those threats. This project absolutely met my goals. I was able to simulate realistic attacks, observe GuardDuty generate high-severity findings, and validate malware protection using a test file. It gave me confidence in my understanding of cloud security monitoring and incident detection, and it clearly strengthened my portfolio with practical, job-ready security skills.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Project Setup

To set up for this project, I deployed a CloudFormation template that launches an intentionally vulnerable cloud environment for security testing and monitoring. The three main components are a publicly accessible EC2 instance hosting a vulnerable web application, an S3 bucket containing sensitive data to simulate a real data-exposure target, and the required IAM roles and security configurations that allow AWS services like GuardDuty to monitor, detect, and analyze malicious activity within the environment.

The web app deployed is called Damn Vulnerable Web Application (DVWA). To practice my GuardDuty skills, I will intentionally interact with and exploit the vulnerabilities in the application—such as weak authentication and insecure configurations—to simulate real-world attack behavior, then observe, analyze, and investigate how Amazon GuardDuty detects and reports these malicious activities in the AWS environment.

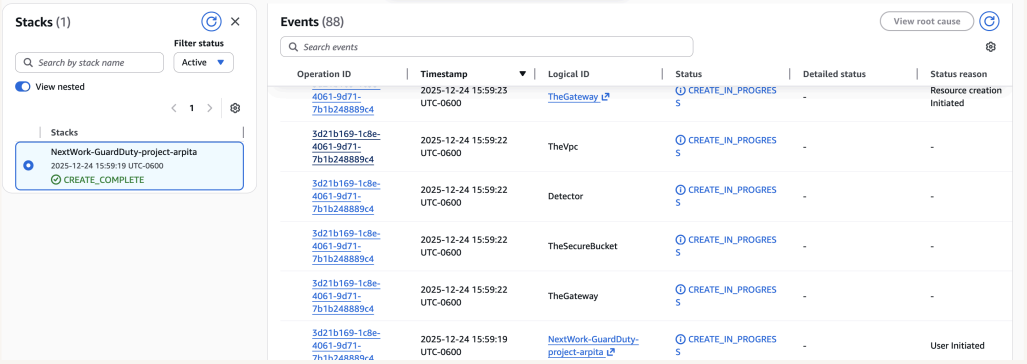
GuardDuty is a fully managed AWS threat detection service that continuously monitors AWS accounts and workloads for malicious activity and unauthorized behavior. In this project, it will analyze logs from AWS CloudTrail, VPC Flow Logs, and DNS activity to detect suspicious actions such as credential compromise, unusual API calls, and potential malware activity, helping identify and investigate security threats in the deployed environment.

CH

Arpita Chowdhury

NextWork Student

nextwork.org



CH
Arpita Chowdhury
NextWork Student
nextwork.org

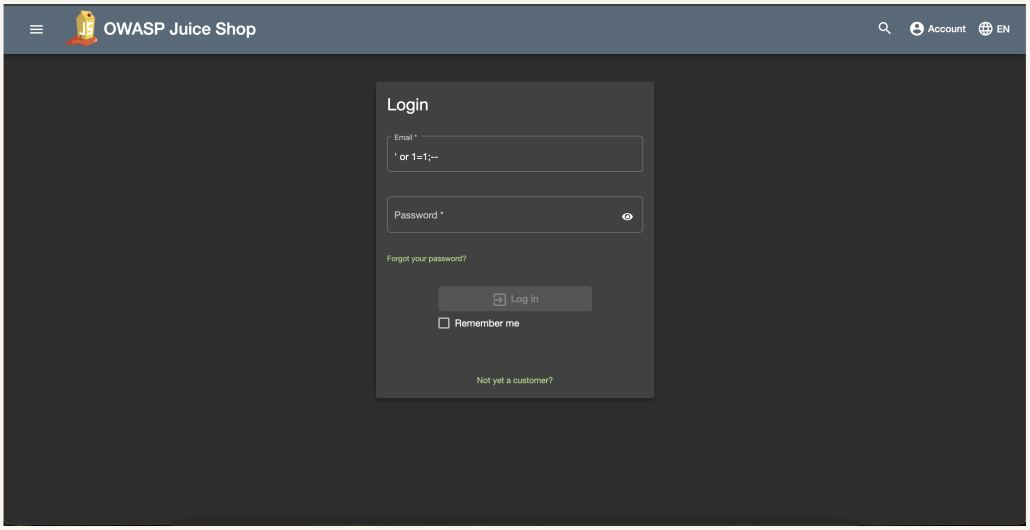
SQL Injection

The first attack I performed on the web app is SQL injection, which means an attacker inserts malicious SQL statements into an application's input fields to manipulate the database behind the web application. SQL injection is a security risk because it can allow unauthorized access to sensitive data, bypass authentication, modify or delete database records, and in severe cases give attackers control over the underlying system.

My SQL injection attack involved entering crafted input into the web application's login fields to manipulate the SQL query executed by the database. This means that instead of the application safely checking a username and password, my input altered the logic of the query so it always evaluated as true (for example, using ' OR 1=1;--), allowing me to bypass authentication and log in without valid credentials.

CH

Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student
nextwork.org

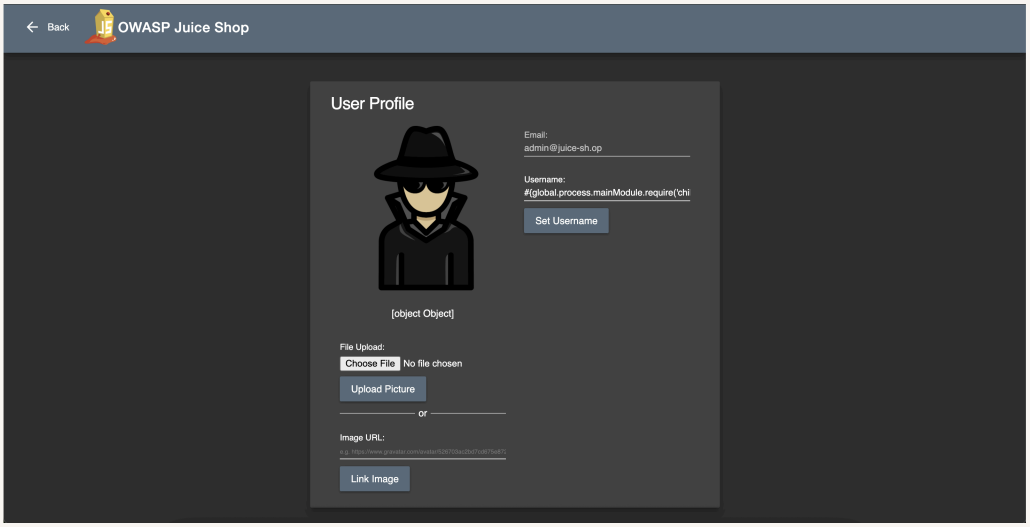
Command Injection

Next, I used command injection, which is a type of security vulnerability where an attacker is able to execute arbitrary operating system commands on a server through a vulnerable application. The Juice Shop web app is vulnerable to this because it does not properly validate or sanitize user input before passing it to system-level commands, allowing malicious input to be interpreted and executed by the underlying operating system instead of being treated as plain data.

To run command injection, I submitted specially crafted input into a vulnerable field of the web application that was passed directly to the underlying system shell instead of being safely handled as data. The script will be interpreted by the operating system as executable commands rather than normal user input, allowing actions to run on the server that the application never intended—demonstrating how improper input validation can lead to serious security compromises. This was done only in a controlled, intentionally vulnerable lab environment to observe how such behavior is detected and reported by Amazon GuardDuty, reinforcing the importance of secure input handling and threat detection in real-world cloud systems.

CH

Arpita Chowdhury
NextWork Student



Arpita Chowdhury
NextWork Student

Attack Verification

To verify the attack's success, I checked whether the command injection allowed me to access sensitive AWS metadata from the compromised EC2 instance. The credentials page showed me valid temporary AWS IAM credentials including an Access Key ID, Secret Access Key, session token, and expiration time which confirmed that the attack successfully retrieved security credentials from the instance metadata service and demonstrated a real credential compromise scenario detected later by Amazon GuardDuty.

```
{
  "AccessKeyId": "ASIAQDF22SWUEKBGXPMP",
  "Code": "Success",
  "Expiration": "2025-12-25T04:37:20Z",
  "LastUpdated": "2025-12-24T22:02:46Z",
  "SecretAccessKey": "9XU9g9wA2o+GPgzEN5I5cYKNnfkQ1D7holzGQhjx",
  "Token":
"IQoJb3JpZ2luX2VjEGYaCXVzLWVhc3QtMSJHMEUCIBYGoYow1XFGiCLbGGGriC3+0b5elJL4K/uoW15pwbbjAiEAq4OBfRcC9QBL8HS+4WnFDB3NjhKkxNaJE9TixgmxPqAquQUILxAAGgwwMDY4MzQyNjM0NjQiDLZF78wX7JT+4x+YniqWBZbPHf
pRKEIDBKTTUEFU+1kgMg/S8fA7EeK9gFRcNteh7Q15DgWAWL/F4Kah3x/jQA3weyTlCTQrU04k7+w+FlW6VQybsymWT0tg0UGYeF+jX4KjuhXK22MUylRurFtasLiUzdUmSNdF7JCBvYLEw9ITYGJr2n48sI4L3X+A/niaLxr49mrn85whjkJJtAH7
QVjsbIoFc3+njkVPClfBuTi822LtlnJO9ArsNKnlr/rw2c29hnVqzRjgL4rlltl1E7ODn2gqBO15hJdYYoYzPXiabc4my0L5fQLHWE4VYBtyYBA6uCI12PE266gZXZE7qcTiM1r0ekJhSwLksB2RL4zJ5iD1L0GG3OkaHBo
7EmLH4WNL36olVidEfpTP+Wskg9ZDENQ9H5n85/CLi0HpezLwh10bPr99Iup4B52VgVTQ0TlEqLBJK3Nl/RxD4gANuZ3PC8wE+yGNbMXeNU9bfymRef55yN3C9nkiI1vMOlKwTI1VO7YTqtkieJERPRW06BM23XY2wKJV1oRgziKoN6VH+VK8G0fUE
TIDeUsENLvT6KW7k0WS2ZzMRiEiELvYVEx6IdlwtOicSz2EYiq8PYyqWX/c9IVtNTi5Gcd+5tf2FRC5DNMOlpAF0TS+F5BKWcSPcXbVliLAMBQnrFMTi0OzNHqKQORxeM09AGtmV1dfBO9QxYY+8VzxrfKrvdqVeFFdWFhYHVMSS/job1pCvdvJLSR7
/XPfUj4o2jQg4YSe+hSKlvDLJS5bbUGII0Pvv+J+T6bptdpc+lp4yr8sT0u8/u+6WMF4X4f2kDEFRmQ3QH4m3ZVDxpfJVokoiJ1Y32dz2hL6QaoziQRqomKIFJqH1PsPd4MOzFscoGOrEByb7Km8KheLR+iNwGWHnNvzlh0u0XL3tga29vmP+C7ILQn
9+7FlzmKqvW7OXI3kATOL6NOH/y0ernml0i/t73yn+KdyjVQLd20HSlSRfGtfLvXMGur/MklmSBTvoMWRv4PFLTFBgppmb3YtuUIaIAKeQ9q54hWWNLlpuimbWP+UHvsExk2Cd2wH+ZZlbTNt7oleUnAAf9aENXxutQiXhysYXaXeHQPWg20I0cuOW2
00Yo",
  "Type": "AWS-HMAC"
}
```

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Using CloudShell for Advanced Attacks

The attack continues in CloudShell, because it provides a browser-based, preconfigured AWS CLI environment that lets me safely test the stolen credentials without setting up tools locally, making it easy to simulate how an attacker could immediately use compromised credentials to access AWS resources and demonstrate the real-world impact of credential theft detected by Amazon GuardDuty.

In CloudShell, I used wget to download the exposed credentials file from the vulnerable Juice Shop web application so I could retrieve the stolen AWS access keys locally. Next, I ran a command using cat and jq to display and neatly format the JSON contents of the file, allowing me to clearly view and extract the Access Key ID, Secret Access Key, and session token needed to configure the AWS CLI and continue the attack scenario.

I then set up a profile, called stolen, to authenticate to AWS using the compromised credentials extracted from the vulnerable web application. I had to create a new profile because the default CloudShell profile uses my legitimate Management Console permissions, and using it would not accurately simulate an attacker’s behavior. By creating a separate profile with the stolen credentials, I can act as a compromised identity and demonstrate how an attacker would access AWS resources, allowing me to observe whether Amazon GuardDuty detects this suspicious activity.

```
~ $ aws configure set profile.stolen.region us-west-2
~ $ aws configure set profile.stolen.aws_access_key_id `cat credentials.json | jq -r '.AccessKeyId'`
~ $ aws configure set profile.stolen.aws_secret_access_key `cat credentials.json | jq -r '.SecretAccessKey'`
~ $ aws configure set profile.stolen.aws_session_token `cat credentials.json | jq -r '.Token'`
~ $ aws s3 cp s3://$JUICESHOPS3BUCKET/secret-information.txt . --profile stolen
download: s3://nextwork-guardduty-project-arpita-thesecurebucket-9csuv12svlx9/secret-information.txt to ./secret-information.txt
~ $ cat secret-information.txt
Dang it - if you can see this text, you're accessing our private information!
~ $
```

CH
Arpita Chowdhury
NextWork Student

GuardDuty's Findings

After performing the attack, GuardDuty reported a finding within a few minutes. Findings are security alerts generated by Amazon GuardDuty that indicate detected suspicious or malicious activity, confirming that the credential compromise and unauthorized access were successfully identified by the threat detection service.

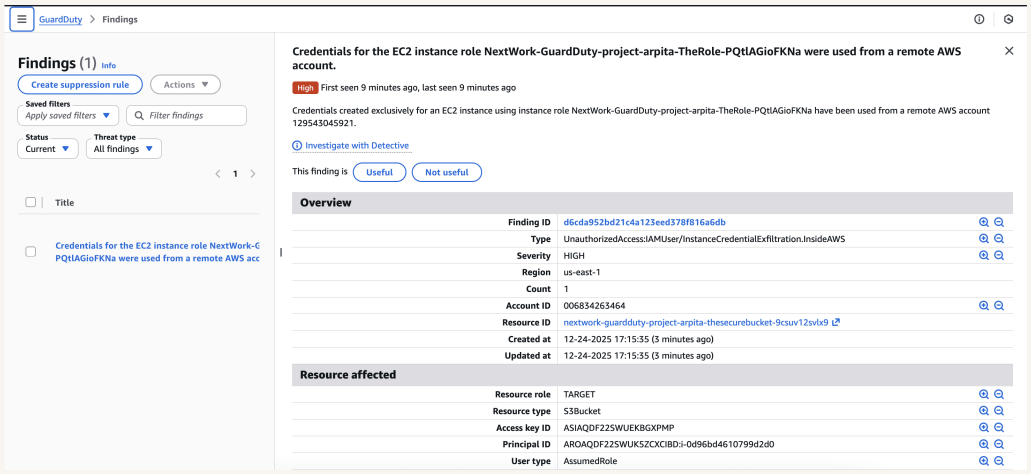
GuardDuty’s finding was called “Credentials for the EC2 instance role were used from a remote AWS account”, which means that temporary IAM credentials created for an EC2 instance were exfiltrated and then used outside of their expected environment. Anomaly detection was used because GuardDuty identified unusual behavior instance role credentials being accessed from a different AWS account than normal which strongly indicates credential compromise and unauthorized access rather than legitimate activity.

GuardDuty’s detailed finding reported that credentials generated for the EC2 instance’s IAM role were exfiltrated and then used from a remote AWS account, indicating unauthorized access. The finding highlighted that instance role credentials meant to be used only by the EC2 instance were observed performing actions outside their expected environment, which is a strong signal of credential compromise and malicious activity.

CH

Arpita Chowdhury

NextWork Student



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Extra: Malware Protection

For my project extension, I enabled Amazon GuardDuty Malware Protection for S3 and compute workloads. Malware is malicious software designed to disrupt systems, steal data, or gain unauthorized access, and enabling this protection allows GuardDuty to automatically scan for known and suspicious malware files and generate findings when a threat is detected.

To test Malware Protection, I uploaded the EICAR test file to the S3 bucket. The uploaded file won't actually cause damage because it is a harmless, industry-standard test string specifically designed to trigger malware detection systems without containing real malicious code, allowing GuardDuty to safely verify that malware protection is working as expected.

Once I uploaded the file, GuardDuty instantly triggered a High-severity malware finding identifying the S3 object as an EICAR test file (Object:S3/MaliciousFile). This verified that GuardDuty's Malware Protection was correctly enabled and functioning as expected, successfully scanning new S3 uploads and detecting potentially malicious content in near real time.

CH

Arpita Chowdhury

NextWork Student